‘

Experiment No.: 08
Title: Hyper parameter tuning in Reinforcement Learning

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 08**

Aim: To study, configure, and evaluate hyperparameter tuning techniques (such as adjusting learning rate, discount factor, and exploration strategies) on Reinforcement Learning models to achieve optimal convergence and maximize cumulative policy rewards.

Resources needed: Any programming language.

Pre Requisite: Machine learning concepts, Statistics

Theory:

1. Role of Hyperparameters in Reinforcement Learning

Unlike supervised learning where dataset labels are static, RL algorithms deal with non-stationary data generated by an agent's evolving policy. Consequently, RL performance is highly sensitive to hyperparameter choices; sub-optimal configurations can cause policy collapse, sample inefficiency, or total failure to converge.

2. Key RL Hyperparameters

- **Learning Rate (α):** Controls the step size for weight/Q-table updates. A value too high causes instability, while a value too low leads to exceedingly slow convergence.

- **Discount Factor (γ):** Determines the present value of future rewards. $\gamma \to 0$ prioritizes immediate gains, while $\gamma \to 1$ optimizes for long-term goal realization.
- **Exploration Rate (ϵ) & Decay:** Governs the trade-off between random exploration and policy exploitation over time.
- **Batch Size & Buffer Capacity:** In deep RL, the replay buffer size and mini-batch sample size dictate temporal correlation reduction and memory efficiency.

3. Automated Hyperparameter Optimization Strategies

- **Grid Search:** Systematically evaluates every combination over a pre-defined discrete parameter grid.
- **Random Search:** Randomly samples hyperparameter configurations from probability distributions; often more sample-efficient than grid search.
- **Bayesian Optimization (Optuna / Ray Tune):** Uses sequential model-based optimization to construct a probabilistic model of the objective function, picking new hyperparameter configurations that maximize expected improvement.

Procedure:

1.**Setup Workspace & Install Dependencies:**
Initialize the Python workspace and install required packages for RL training and automated hyperparameter optimization.

1. Bash

pip install gymnasium numpy optuna matplotlib

2. **Define the Base RL Environment & Agent:**

Create a standard environment (e.g., CartPole-v1 or FrozenLake-v1) and instantiate a parameterized RL agent (e.g., Q-Learning or DQN).

3. **Construct the Optimization Objective Function:**

Wrap the agent training loop inside an objective function that accepts a hyperparameter trial instance. Configure the trial to sample hyperparameters dynamically:

- Learning rate $\alpha \in [10^{-4}, 10^{-1}]$ (log-scale)
- Discount factor $\gamma \in [0.90, 0.999]$
- Exploration decay rate $\in [10^{-4}, 10^{-2}]$

4. **Execute Trial Searches using Optuna:**

Run an automated study across multiple trials (e.g., 50 trials) to maximize the mean cumulative reward evaluated over validation episodes.

5. Python

import optuna

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
    gamma = trial.suggest_float("gamma", 0.9, 0.999)
    # Train agent with (lr, gamma) and return mean reward
    return evaluate_agent(lr, gamma)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
```

6. **Analyze Results & Parameter Sensitivity:**

Extract the best parameter combinations and generate parallel coordinate and hyperparameter importance plots to observe sensitivity impacts on performance.

Results: (Steps with screenshots)

```
import json
import numpy as np
import pandas as pd
import gymnasium as gym
import optuna

from gymnasium import spaces
from optuna.visualization import plot_parallel_coordinate, plot_param_importances

df = pd.read_csv("india_2000_2024_daily_weather(1).csv")
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values(["city", "date"])

train_parts = []
val_parts = []

for _, group in df.groupby("city", sort=False):
    group = group.sort_values("date").copy()
    group["target"] = (
        group["temperature_2m_max"].shift(-1) >= 40
    ).astype(float)
    group = group.iloc[:-1].copy()

    split = int(len(group) * 0.8)
    train_parts.append(group.iloc[:split])
    val_parts.append(group.iloc[split:])

train_df = pd.concat(train_parts, ignore_index=True)
val_df = pd.concat(val_parts, ignore_index=True)

features = [
    "temperature_2m_max",
    "apparent_temperature_max",
```

```
    "precipitation_sum",
    "wind_speed_10m_max",
    "date"
]

def prepare_data(data):
    result = data.copy()
    result["month"] = result["date"].dt.month

    cols = [
        "temperature_2m_max",
        "apparent_temperature_max",
        "precipitation_sum",
        "wind_speed_10m_max"
    ]

    result[cols] = result[cols].fillna(0)

    return result.reset_index(drop=True)

train_df = prepare_data(train_df)
val_df = prepare_data(val_df)

class WeatherEnv(gym.Env):
    def __init__(self, data, max_steps=365, random_start=False):
        super().__init__()

        self.data = data.reset_index(drop=True)
        self.max_steps = max_steps
        self.random_start = random_start

        self.temps = self.data["temperature_2m_max"].to_numpy()
        self.apparent = self.data["apparent_temperature_max"].to_numpy()
        self.precip = self.data["precipitation_sum"].to_numpy()
        self.wind = self.data["wind_speed_10m_max"].to_numpy()
        self.month = self.data["month"].to_numpy()
        self.target = self.data["target"].to_numpy()
        self.cities = self.data["city"].to_numpy()

        self.action_space = spaces.Discrete(2)
        self.observation_space = spaces.Box(
            low=-np.inf,
            high=np.inf,
            shape=(5,),
            dtype=np.float32
        )

    def observation(self, index):
        return np.array([
            self.temps[index],
            self.apparent[index],
            self.precip[index],
            self.wind[index],
```

```
            self.month[index]
        ], dtype=np.float32)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)

        if options and "start_index" in options:
            self.index = int(options["start_index"])
        elif self.random_start:
            self.index = int(self.np_random.integers(len(self.data)))
        else:
            self.index = 0

        self.start_city = self.cities[self.index]
        self.steps = 0

        return self.observation(self.index), {}

    def step(self, action):
        actual = int(self.target[self.index])
        reward = (
            2.0 if action == 1 and actual == 1 else
            -1.0 if action == 1 and actual == 0 else
            1.0 if action == 0 and actual == 0 else
            -2.0
        )

        self.steps += 1
        next_index = self.index + 1

        terminated = (
            next_index >= len(self.data)
            or self.steps >= self.max_steps
            or self.cities[next_index] != self.start_city
            if next_index < len(self.data)
            else True
        )

        if next_index >= len(self.data):
            terminated = True

        if not terminated:
            self.index = next_index
            next_observation = self.observation(self.index)
        else:
            next_observation = self.observation(self.index)

        return next_observation, reward, terminated, False, {}


def discretize(obs):
    return (
        np.digitize(obs[0], [25, 30, 35, 40, 45]),
```

```
        np.digitize(obs[1], [25, 30, 35, 40, 45]),
        np.digitize(obs[2], [0.1, 1, 5, 10]),
        np.digitize(obs[3], [10, 20, 30, 40]),
        np.digitize(obs[4], [3, 6, 9])
    )


def train_agent(env, lr, gamma, decay, episodes=30):
    q_table = np.zeros((6, 6, 5, 5, 4, 2))
    epsilon = 1.0

    for _ in range(episodes):
        obs, _ = env.reset()
        done = False

        while not done:
            state = discretize(obs)

            if np.random.random() < epsilon:
                action = env.action_space.sample()
            else:
                action = int(np.argmax(q_table[state]))

            next_obs, reward, terminated, truncated, _ = env.step(action)
            next_state = discretize(next_obs)

            q_table[state + (action,)] += lr * (
                reward
                + gamma * np.max(q_table[next_state])
                - q_table[state + (action,)]
            )

            obs = next_obs
            done = terminated or truncated
            epsilon = max(0.01, epsilon * (1 - decay))

    return q_table


def evaluate_agent(q_table, data):
    env = WeatherEnv(data, max_steps=365)
    rewards = []

    for city in data["city"].unique():
        indices = np.flatnonzero(data["city"].to_numpy() == city)

        for start in indices[::365]:
            obs, _ = env.reset(options={"start_index": int(start)})
            total_reward = 0
            done = False

            while not done:
                state = discretize(obs)
```

```
                action = int(np.argmax(q_table[state]))

                obs, reward, terminated, truncated, _ = env.step(action)
                total_reward += reward
                done = terminated or truncated

            rewards.append(total_reward)

    return float(np.mean(rewards)) if rewards else 0.0


def objective(trial):
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
    gamma = trial.suggest_float("gamma", 0.90, 0.999)
    decay = trial.suggest_float("decay", 1e-4, 1e-2, log=True)

    env = WeatherEnv(train_df, random_start=True)
    q_table = train_agent(env, lr, gamma, decay)
    return evaluate_agent(q_table, val_df)


study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42)
)

study.optimize(objective, n_trials=50)

best_params = study.best_params

with open("best_hyperparameters.json", "w") as file:
    json.dump(best_params, file, indent=4)

best_q_table = train_agent(
    WeatherEnv(train_df, random_start=True),
    best_params["lr"],
    best_params["gamma"],
    best_params["decay"],
    episodes=30
)

validation_reward = evaluate_agent(best_q_table, val_df)

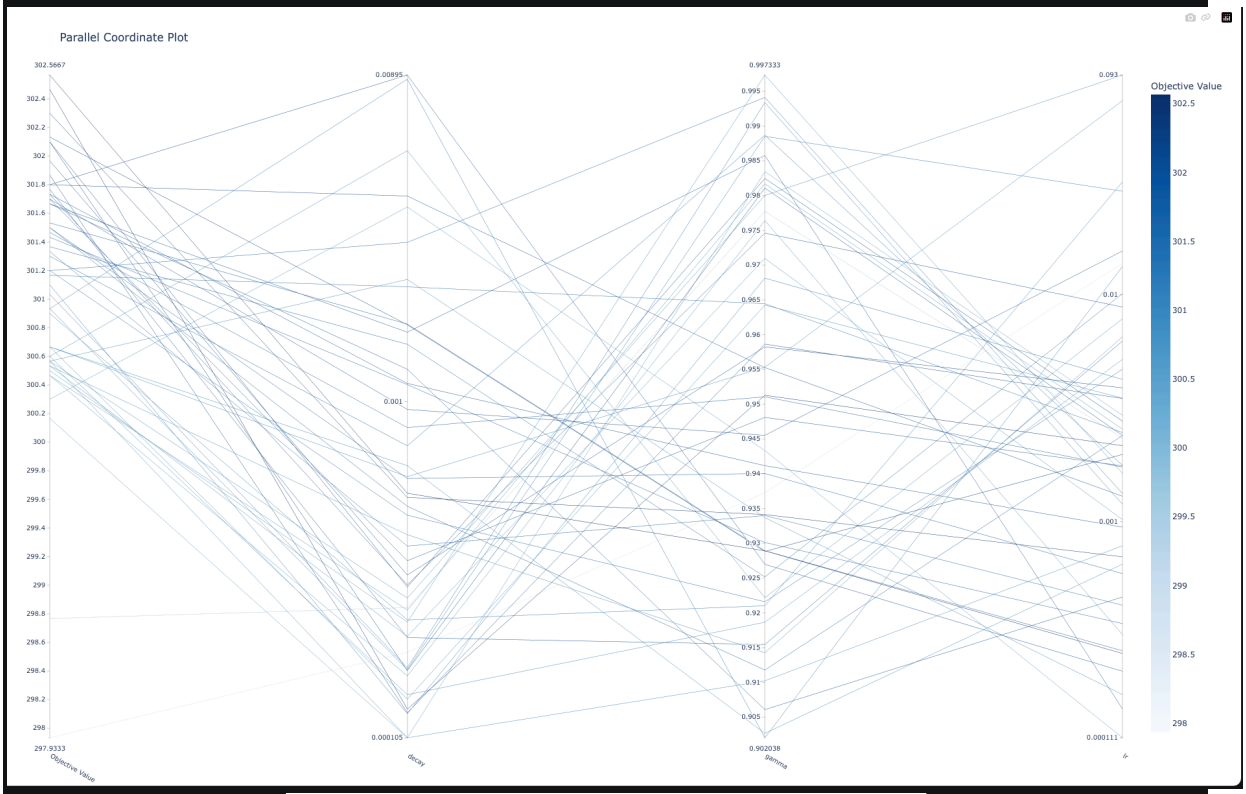
plot_parallel_coordinate(study).write_html("parallel_coordinate.html")
plot_param_importances(study).write_html("parameter_importance.html")

np.save("best_q_table.npy", best_q_table)

print("Best parameters:", best_params)
print("Best Optuna reward:", study.best_value)
print("Final validation reward:", validation_reward)
```

OUTPUT:

```
Best parameters: {'lr': 0.00026210878782654407, 'gamma': 0.9289223202049867,
'decay': 0.0005404103854647331}
Best Optuna reward: 302.56666666666666
Final validation reward: 300.43333333333334
```





Questions:

1. Why are Reinforcement Learning algorithms significantly more sensitive to hyperparameter choices than Supervised Learning models?

Reinforcement Learning (RL) algorithms are generally more sensitive to hyperparameter choices because they learn through interaction with an environment using rewards and penalties. Parameters such as learning rate, discount factor, and exploration rate directly influence the agent's learning behaviour, exploration of actions, and long-term decision-making. Small changes in these parameters can affect the agent's experience, reward

distribution, and convergence. Unlike supervised learning, where models learn from a fixed dataset with known target outputs, RL agents generate their own training experiences through interaction with the environment. Consequently, poor hyperparameter settings may lead to unstable learning, slow convergence, or suboptimal policies. Therefore, careful hyperparameter tuning is essential for achieving reliable performance in Reinforcement Learning.

2. Compare Grid Search, Random Search, and Bayesian Optimization in terms of sample efficiency and computational complexity.

Grid Search evaluates every possible combination of predefined hyperparameter values, making it computationally expensive when the number of parameters or candidate values increases. Although it systematically explores the search space, it may waste resources on less important parameters. Random Search selects hyperparameter combinations randomly and often achieves better results with fewer trials because it explores a wider variety of values without evaluating every combination. Bayesian Optimization is generally more sample-efficient because it uses the results of previous trials to select promising hyperparameter combinations for subsequent evaluations. However, it requires additional computational resources to build and update its probabilistic model. Grid Search is suitable for small search spaces, Random Search is useful for larger spaces, and Bayesian Optimization is particularly effective when each model-training evaluation is computationally expensive.

Conclusion: (Conclusion to be based on the objectives and outcomes achieved)

Hence we performed the experiment for successfully.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Websites:

1. *Reinforcement Learning: An Introduction* — Richard S. Sutton and Andrew G. Barto, MIT Press.

2. *Gymnasium Documentation* — gymnasium.farama.org